

Medical GenAI RAG Platform

Step-by-Step Development Plan
14 Build Phases — Folder Structure & File Index (No Source Code)

DOCUMENT TYPE	Project Development Plan
COMPANION DOCUMENT	Solution Architecture Design Document (v1.0)
VERSION	1.0 — Draft for Review
DATE	10 August 2026
SCOPE	14 sequential build phases, folder structure and file names only
EXPLICITLY EXCLUDED	No source code, no implementation snippets — structure and plan only

Table of Contents

1. Document Control

2. Purpose of This Document

3. Development Methodology

4. Target Project Structure (End State)

5. Phase-by-Phase Development Plan

Phase 1 — Environment Setup

Phase 2 — Qdrant Setup

Phase 3 — Ollama Setup

Phase 4 — Embedding Service

Phase 5 — Document Ingestion

Phase 6 — Chunking + Metadata

Phase 7 — Qdrant Indexing

Phase 8 — Basic Retrieval

Phase 9 — RAG Pipeline

Phase 10 — Grounding + Guardrails

Phase 11 — FastAPI

Phase 12 — Streamlit UI

Phase 13 — Evaluation

Phase 14 — Future Agent Integration Design

6. Master File Checklist

7. Test File Index

1. Document Control

Field	Value
Document Title	Medical GenAI RAG Platform — Project Development Plan
Version	1.0 — Draft for Review
Date	10 August 2026
Owner	Generative AI Architecture Team
Companion Documents	Solution Architecture Design Document v1.0 (target architecture and rationale)
Scope	The 14-phase build plan for the RAG platform's own codebase. Deployment infrastructure (Azure) is covered by the Solution Architecture Design Document, Section 10.

2. Purpose of This Document

This document is the execution plan for building the Medical GenAI RAG Platform described in the Solution Architecture Design Document. It exists to answer one question at every point in the build: **which files get created or touched next, and why**. It intentionally contains **no source code** — only folder structure, file names, and one-line descriptions of what each file is responsible for. Implementation detail belongs in the codebase itself; this document is the map, not the territory.

The plan is organized into **14 sequential phases**, matching the implementation order mandated for this project: each phase is explained, built, and verified before the next begins. Every phase section below lists exactly which files are new or modified, what depends on what, and the concrete definition of done that closes the phase out.

3. Development Methodology

3.1 The Five-Step Loop

Every phase in Section 5 follows the same five steps, in order:

- **Explain** — what is being built in this phase and why, in the context of the phase before and after it.
- **Implement** — create or modify only the files listed for that phase.
- **Wire to configuration** — every new module reads from the centralized settings established in Phase 1; no values are hard-coded.
- **Verify** — run the test file(s) for that phase and confirm the phase's definition of done.
- **Checkpoint** — update the master file checklist (Section 6) before moving to the next phase.

3.2 Phase Dependency Flow

Phase 1 is the only universal prerequisite. Phases 2, 3, and 4 have no dependency on one another and may be built in any order — they are sequenced 2 → 3 → 4 only to match the architecture document's presentation order. From Phase 5 onward the plan is strictly sequential: each phase consumes the previous phase's output.

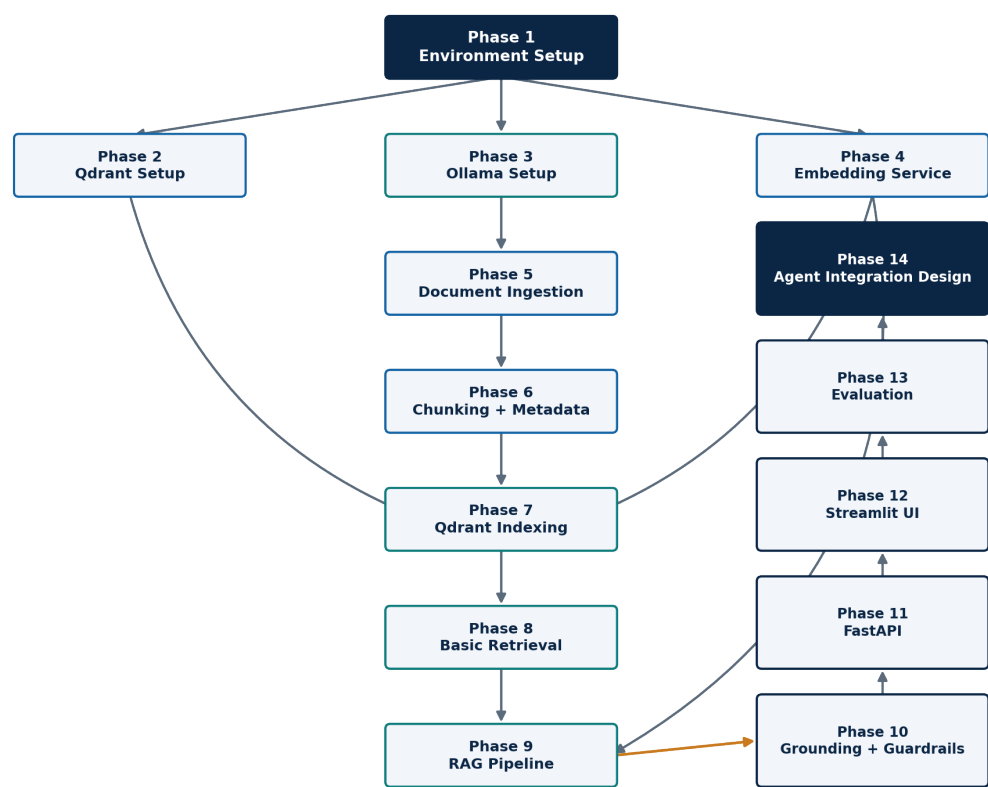


Figure 1 — Phase dependency flow. Phases 2/3/4 branch from Phase 1 independently; Phase 7 is the join point where ingestion, chunking, and embeddings converge into the vector store.

3.3 Legend Used in This Document

Marker	Meaning
NEW	File is created for the first time in this phase
MODIFIED	File already exists (typically as an empty placeholder from Phase 1) and is filled in or extended in this phase

Every module in the codebase exists as an empty, importable placeholder from Phase 1 onward (see Section 4). This means the file tree never changes shape after Phase 1 — only file contents change, phase by phase. The NEW / MODIFIED distinction below reflects when a file's real logic is written, not when the file path is first created on disk.

4. Target Project Structure (End State)

This is the complete folder structure once all 14 phases are finished. Section 5 shows, phase by phase, exactly when each file's logic is written.

```
medical-rag/  
|-- app/  
|   |-- main.py  
|   |-- api/  
|       |-- health.py  
|       |-- documents.py  
|       |-- rag.py  
|   |-- ingestion/  
|       |-- loaders.py  
|       |-- parser.py  
|       |-- chunker.py  
|       |-- metadata.py  
|   |-- embeddings/  
|       |-- embedding_service.py  
|   |-- vectorstore/  
|       |-- qdrant_store.py  
|   |-- retrieval/  
|       |-- retriever.py  
|       |-- reranker.py  
|   |-- llm/  
|       |-- ollama_service.py  
|   |-- rag/  
|       |-- rag_pipeline.py  
|       |-- context_builder.py  
|       |-- grounding.py  
|   |-- guardrails/  
|       |-- safety.py  
|       |-- prompt_injection.py  
|   |-- models/  
|       |-- requests.py  
|       |-- responses.py  
|   |-- config/  
|       |-- settings.py  
|-- ui/  
|   |-- streamlit_app.py  
|-- eval/  
|   |-- evaluate.py  
|   |-- metrics.py  
|   |-- dataset.py  
|-- docs/  
|   |-- agent_integration_design.md
```

```
|  `-- agent_tool_contract.md
|-- data/
|   |-- raw/
|   |-- processed/
|   `-- sample_documents/
|-- tests/
|   |-- test_config.py
|   |-- test_ingestion.py
|   |-- test_chunking.py
|   |-- test_embeddings.py
|   |-- test_qdrant_indexing.py
|   |-- test_qdrant_retrieval.py
|   |-- test_metadata_filtering.py
|   |-- test_reranking.py
|   |-- test_ollama_generation.py
|   |-- test_grounding.py
|   |-- test_citations.py
|   |-- test_prompt_injection.py
|   `-- test_conflicting_documents.py
|-- scripts/
|   |-- ingest.py
|   `-- index.py
|-- docker-compose.yml
|-- requirements.txt
|-- .env.example
`-- README.md
```

5. Phase-by-Phase Development Plan

PHASE 1

Environment Setup

Objective: Establish the project skeleton, dependency list, and a single centralized configuration module that every later phase reads from.

Files

Status	File	Purpose
NEW	requirements.txt	Pinned, local/open-source Python dependencies
NEW	.env.example	Every configurable value: Qdrant, Ollama, embedding model, chunking, retrieval
NEW	docker-compose.yml	Local Qdrant service definition
NEW	app/config/settings.py	Centralized, cached settings object used by every module
NEW	app/__init__.py and one per subpackage	Makes every planned module importable from day one
NEW	tests/test_config.py	Verifies settings load correctly and are cached as a singleton
NEW	README.md	Setup commands, phase log, troubleshooting
NEW	data/raw/, data/processed/, data/sample_documents/	Empty data directories (placeholders) for later phases

Dependencies

None — this is the starting point for the entire build.

Definition of Done

- Virtual environment created and requirements installed without error
- app/config/settings.py loads values from .env correctly
- tests/test_config.py passes

PHASE 2

Qdrant Setup

Objective: Stand up a local Qdrant instance and implement collection management and health checks.

Files

Status	File	Purpose
MODIFIED	app/vectorstore/qdrant_store.py	create_collection, health_check, get_collection_stats implemented

Dependencies

Phase 1 (settings module, docker-compose.yml).

Definition of Done

- Qdrant container starts successfully from docker-compose.yml
- medical_knowledge collection is created with the correct vector dimension
- Health check reports a healthy, reachable collection

PHASE 3

Ollama Setup

Objective: Validate the local Ollama runtime and implement the LLM service abstraction used by every later generation step.

Files

Status	File	Purpose
MODIFIED	app/llm/ollama_service.py	generate, generate_structured, health_check implemented

Dependencies

Phase 1 (settings module for OLLAMA_BASE_URL / OLLAMA_MODEL).

Definition of Done

- A model is pulled and running locally in Ollama
- health_check() confirms connectivity to the Ollama runtime
- generate() returns a text completion for a test prompt

PHASE 4

Embedding Service

Objective: Implement the local Sentence Transformers embedding service shared identically by indexing and querying.

Files

Status	File	Purpose
MODIFIED	app/embeddings/embedding_service.py	embed_text, embed_documents, embed_query implemented
MODIFIED	tests/test_embeddings.py	Confirms vector dimension and shared-model behavior

Dependencies

Phase 1 (settings module for EMBEDDING_MODEL).

Definition of Done

- Embedding vector dimension matches the collection created in Phase 2
- The same model instance is confirmed to serve both indexing and query embedding
- tests/test_embeddings.py passes

PHASE 5

Document Ingestion

Objective: Implement format-specific loaders and the cleaning/normalization stage for PDF, DOCX, TXT, Markdown, and JSON sources.

Files

Status	File	Purpose
MODIFIED	app/ingestion/loaders.py	Format-specific extraction for PDF, DOCX, TXT, Markdown, JSON
MODIFIED	app/ingestion/parser.py	Whitespace/encoding normalization, boilerplate stripping
MODIFIED	scripts/ingest.py	CLI: data/raw/ → data/processed/
NEW	data/sample_documents/*	Synthetic demo corpus (10+ documents per architecture Section 4.1)
MODIFIED	tests/test_ingestion.py	One test per supported format

Dependencies

Phase 1 (project skeleton and data/ directories).

Definition of Done

- scripts/ingest.py successfully processes a document of each supported format
- Cleaned output appears in data/processed/
- tests/test_ingestion.py passes for every format

PHASE 6

Chunking + Metadata

Objective: Implement fixed-size chunking and metadata extraction, producing chunk objects ready for embedding.

Files

Status	File	Purpose
MODIFIED	app/ingestion/chunker.py	Configurable fixed-size chunking (CHUNK_SIZE / CHUNK_OVERLAP)
MODIFIED	app/ingestion/metadata.py	Builds the standard metadata payload per chunk
MODIFIED	tests/test_chunking.py	Verifies chunk size/overlap behavior
MODIFIED	tests/test_metadata_filtering.py	Verifies metadata schema correctness (filtering itself is tested in Phase 8)

Dependencies

Phase 5 (cleaned text to chunk).

Definition of Done

- Chunk size and overlap are configurable via settings, not hard-coded
- Metadata payload matches the schema defined in the architecture document (Section 8.1)
- tests/test_chunking.py passes

PHASE 7

Qdrant Indexing

Objective: Wire ingestion, chunking, and embeddings into Qdrant via a single indexing pipeline.

Files

Status	File	Purpose
MODIFIED	app/vectorstore/qdrant_store.py	upsert_vectors (single and batch) implemented
MODIFIED	scripts/index.py	CLI: data/processed/ → Qdrant
MODIFIED	tests/test_qdrant_indexing.py	Verifies upsert and resulting collection statistics

Dependencies

Phase 2 (Qdrant), Phase 4 (embeddings), Phase 6 (chunks + metadata) — this is the join point where all three converge.

Definition of Done

- scripts/index.py successfully indexes the full demo corpus into Qdrant
- Collection statistics show the expected point count
- tests/test_qdrant_indexing.py passes

PHASE 8

Basic Retrieval

Objective: Implement semantic retrieval with metadata filtering and workflow/screen-aware prioritization.

Files

Status	File	Purpose
MODIFIED	app/retrieval/retriever.py	Semantic search + metadata filter + screen-aware ranking
MODIFIED	app/models/requests.py	RAGSearchRequest / RAGQueryRequest schemas defined
MODIFIED	tests/test_qdrant_retrieval.py	Verifies top-K ranking behavior
MODIFIED	tests/test_metadata_filtering.py	Verifies filter + boost behavior end-to-end

Dependencies

Phase 7 (an indexed collection to query against).

Definition of Done

- Retriever returns top-K results ranked by similarity score
- workflow_id / screen_id prioritization verified against sample queries
- tests/test_qdrant_retrieval.py and tests/test_metadata_filtering.py pass

PHASE 9

RAG Pipeline

Objective: Orchestrate retrieval, reranking, context construction, and generation into one pipeline with a defined response schema.

Files

Status	File	Purpose
MODIFIED	app/retrieval/reranker.py	Cross-encoder reranking implemented
MODIFIED	app/rag/context_builder.py	Assembles the context block sent to the LLM
MODIFIED	app/rag/rag_pipeline.py	End-to-end orchestration of the above
MODIFIED	app/models/responses.py	RAGQueryResponse / SourceCitation schemas defined
MODIFIED	tests/test_reranking.py	Verifies reranked ordering and top-K truncation
MODIFIED	tests/test_ollama_generation.py	Verifies generation against a fixed prompt contract
MODIFIED	tests/test_citations.py	Verifies citations map back to real source chunks

Dependencies

Phase 3 (Ollama service), Phase 8 (retriever).

Definition of Done

- A sample query returns an answer with correct citations end-to-end

- tests/test_reranking.py, tests/test_ollama_generation.py, and tests/test_citations.py all pass

PHASE 10

Grounding + Guardrails

Objective: Add grounding assessment, conflict detection, and prompt-injection screening around the pipeline built in Phase 9.

Files

Status	File	Purpose
MODIFIED	app/rag/grounding.py	grounded flag, confidence score, HIGH/MEDIUM/LOW bucket, conflict surfacing
MODIFIED	app/guardrails/safety.py	Response-level safety validation
MODIFIED	app/guardrails/prompt_injection.py	Scans retrieved chunks for embedded instructions
MODIFIED	tests/test_grounding.py	Verifies confidence thresholds and fallback behavior
MODIFIED	tests/test_prompt_injection.py	Verifies injected instructions are detected and neutralized
MODIFIED	tests/test_conflicting_documents.py	Verifies conflicts are surfaced, not silently resolved

Dependencies

Phase 9 (the pipeline this phase wraps trust checks around).

Definition of Done

- Low-confidence queries return the insufficient-evidence fallback rather than a guess
- Conflicting source documents are surfaced with both sources rather than one being silently chosen
- Retrieved chunks containing injected instructions are flagged and not followed
- All three test files in this phase pass

PHASE 11

FastAPI

Objective: Expose the full platform through a FastAPI service — the single integration surface for the UI and the future Agent.

Files

Status	File	Purpose
MODIFIED	app/main.py	FastAPI app instantiation, router registration, startup checks
MODIFIED	app/api/health.py	GET /health, /qdrant/health, /ollama/health
MODIFIED	app/api/documents.py	POST /documents/ingest, /documents/index, GET /documents/{id}
MODIFIED	app/api/rag.py	POST /rag/search, /rag/query

Dependencies

Phases 2 through 10 (every service this layer exposes).

Definition of Done

- The FastAPI app starts successfully and serves all listed endpoints
- Health endpoints report healthy status for Qdrant and Ollama
- POST /rag/query returns a grounded, cited response for a sample question
- POST /documents/ingest and /documents/index process a new document end-to-end via the API

PHASE 12

Streamlit UI

Objective: Build a human-facing demonstration UI that exercises the FastAPI service exactly as an external caller would.

Files

Status	File	Purpose
NEW	ui/streamlit_app.py	Upload, index, ask, and inspect retrieved chunks / answer / citations / confidence / grounding status

Dependencies

Phase 11 (the FastAPI service the UI calls).

Definition of Done

- A document can be uploaded and indexed through the UI
- A question can be submitted and the answer, citations, confidence, and grounding status are all visible
- The UI calls only the FastAPI service — no direct Qdrant or Ollama access from the UI layer

PHASE 13

Evaluation

Objective: Build the RAG evaluation harness measuring retrieval and generation quality against the synthetic demo corpus.

Files

Status	File	Purpose
NEW	eval/dataset.py	20 sample questions with expected answers/sources
NEW	eval/metrics.py	Precision, recall, faithfulness, citation correctness, hallucination rate, latency
NEW	eval/evaluate.py	Runs the dataset through the pipeline and produces a report

Dependencies

Phase 9 (pipeline) and Phase 11 (API, if evaluation runs at the API level rather than in-process).

Definition of Done

- eval/evaluate.py runs the full 20-question demo set without error
- A report is produced covering all metrics defined in the architecture document, Section 14.2

PHASE 14

Future Agent Integration Design

Objective: Document the tool-call contract the future Medical Agent will use. Design artifacts only — no runtime code changes.

Files

Status	File	Purpose
NEW	docs/agent_integration_design.md	How the future Agent will consume this platform (Memory / RAG / Tools separation)
NEW	docs/agent_tool_contract.md	rag.query, rag.search, rag.get_screen_knowledge, rag.get_workflow_instructions, rag.get_validation_rules specifications

Dependencies

All prior phases — this phase documents the platform as built, it does not extend it.

Definition of Done

- Tool contract in docs/agent_tool_contract.md matches the API defined in Phase 11 exactly
- No RAG platform code changes are required for the future Agent's first integration
- Design reviewed and approved before Agent-side work begins

End of the 14-phase plan

Everything from here is Agent-side work — planning, decision-making, and tool orchestration — and is out of scope for this platform by design (Solution Architecture Design Document, Section 4.2). Azure production deployment of the platform built here is covered separately in that document's Section 10.2.

6. Master File Checklist

Every file in the final project structure, the phase that implements it, and its purpose.

File	Phase	Purpose
requirements.txt	1	Pinned local/open-source dependencies
.env.example	1	All configurable environment values
docker-compose.yml	1	Local Qdrant service definition
app/config/settings.py	1	Centralized configuration
tests/test_config.py	1	Settings load + singleton cache
README.md	1	Setup, phase log, troubleshooting
app/vectorstore/qdrant_store.py	2 / 7	Collection mgmt + health (2); upsert (7)
app/llm/ollama_service.py	3	Local LLM runtime client
app/embeddings/embedding_service.py	4	Local embedding generation
tests/test_embeddings.py	4	Embedding dimension + shared-model checks
app/ingestion/loaders.py	5	Per-format document extraction
app/ingestion/parser.py	5	Cleaning / normalization
scripts/ingest.py	5	CLI: raw → processed
data/sample_documents/*	5	Synthetic demo corpus
tests/test_ingestion.py	5	Per-format ingestion checks
app/ingestion/chunker.py	6	Fixed-size chunking
app/ingestion/metadata.py	6	Metadata payload extraction
tests/test_chunking.py	6	Chunk size/overlap checks
tests/test_metadata_filtering.py	6 / 8	Schema checks (6); filter behavior (8)
scripts/index.py	7	CLI: processed → Qdrant
tests/test_qdrant_indexing.py	7	Upsert + collection stats checks
app/retrieval/retriever.py	8	Semantic + metadata-filtered retrieval
app/models/requests.py	8	Request schemas
tests/test_qdrant_retrieval.py	8	Top-K ranking checks
app/retrieval/reranker.py	9	Cross-encoder reranking
app/rag/context_builder.py	9	Context assembly
app/rag/rag_pipeline.py	9	End-to-end orchestration
app/models/responses.py	9	Response schemas
tests/test_reranking.py	9	Reranked order + truncation checks
tests/test_ollama_generation.py	9	Generation contract checks
tests/test_citations.py	9	Citation-to-source mapping checks
app/rag/grounding.py	10	Grounding + confidence + conflict surfacing
app/guardrails/safety.py	10	Response-level safety validation

File	Phase	Purpose
app/guardrails/prompt_injection.py	10	Injected-instruction screening
tests/test_grounding.py	10	Confidence threshold + fallback checks
tests/test_prompt_injection.py	10	Injection detection checks
tests/test_conflicting_documents.py	10	Conflict-surfacing checks
app/main.py	11	FastAPI app entrypoint
app/api/health.py	11	Health endpoints
app/api/documents.py	11	Ingestion/indexing endpoints
app/api/rag.py	11	Search/query endpoints
ui/streamlit_app.py	12	Demonstration UI
eval/dataset.py	13	20-question evaluation set
eval/metrics.py	13	Evaluation metric implementations
eval/evaluate.py	13	Evaluation runner + report
docs/agent_integration_design.md	14	Agent integration design
docs/agent_tool_contract.md	14	Agent tool-call contract

7. Test File Index

Every test file, the phase it belongs to, and what it verifies — the definition-of-done evidence for each phase.

Test File	Phase	Verifies
tests/test_config.py	1	Settings load correctly and are cached as a singleton
tests/test_embeddings.py	4	Embedding vector dimension and shared indexing/query model
tests/test_ingestion.py	5	Correct extraction for PDF, DOCX, TXT, Markdown, JSON
tests/test_chunking.py	6	Chunk size/overlap behavior is respected
tests/test_metadata_filtering.py	6 / 8	Metadata schema correctness; filter + boost behavior
tests/test_qdrant_indexing.py	7	Vectors upsert correctly; collection stats match
tests/test_qdrant_retrieval.py	8	Top-K semantic ranking is correct
tests/test_reranking.py	9	Reranked ordering and top-K truncation
tests/test_ollama_generation.py	9	Generation follows the grounded-RAG prompt contract
tests/test_citations.py	9	Citations map back to real, retrieved source chunks
tests/test_grounding.py	10	Confidence thresholds and insufficient-evidence fallback
tests/test_prompt_injection.py	10	Injected instructions in retrieved content are neutralized
tests/test_conflicting_documents.py	10	Conflicting sources are surfaced, not silently resolved

Every phase closes with a passing test file

No phase in Section 5 is considered complete until its listed test file(s) pass. This test file index is the single place to confirm the full build is verified end to end.